

DivivApp | Informe de Performance

DESARROLLO DE APLICACIONES I

Luka Peralta Perez

LU: 1170204

Android Profiler Report

Campo	Información
Aplicación	DivivApp
Alumno/a	Luka Peralta Perez
Versión analizada	1.0.0

1. Objetivo y alcance

El objetivo del análisis es evaluar el comportamiento de **DivivApp** durante un recorrido representativo de uso. Se observarán el consumo de memoria, la liberación de objetos después de la navegación (Memory Leaks), la actividad de CPU en peticiones asíncronas y la estabilidad de las recomposiciones en Jetpack Compose. El análisis se concentra en las pantallas Splash, Login, Dashboard, Menú, Detalle de Plato, Edición (Admin) y Checkout. Las mediciones deben efectuarse sobre una compilación depurable y con el mismo dispositivo durante todo el procedimiento.

2. Resumen ejecutivo

Dimensión	Resultado	Conclusión breve
Memoria	Uso estable durante el recorrido completo. Recuperación parcial tras GC.	No se observó crecimiento continuo ni consumo anormal de memoria, a pesar de la descarga de imágenes externas.
Heap dump / leaks	No se detectaron instancias retenidas de pantallas cerradas.	No se encontraron evidencias de memory leaks en los ViewModels ni en la MainActivity.
CPU y fluidez	Picos asociados a la consulta inicial a la API de Edamam. Navegación fluida.	El rendimiento fue estable y sin bloqueos prolongados del hilo principal gracias al uso de Coroutines (Dispatchers.IO).
Recomposiciones	Recomposiciones limitadas a componentes afectados por el cambio de estado (Flow).	No se observaron recomposiciones excesivas en estados estáticos de la UI de Compose.

3. Entorno de prueba

Dato	Valor utilizado
Android Studio	Koala / Ladybug (2024.x)
Dispositivo	Emulador Android
Modelo	Medium Phone
Versión de Android / API	Android 15.0 / API 35
RAM asignada	4 GB
Arquitectura	x86_64
Tipo de build	Debug / Profiler con datos completos
Conectividad	Wi-Fi mediante el emulador
Estado inicial	Sesión de Firebase iniciada / Caché limpia / Base de datos Room vacía

4. Recorrido controlado

Para todas las mediciones se utilizó el mismo recorrido funcional. Durante el recorrido no deben registrarse cierres inesperados (Crashes), ANR, crecimiento continuo de memoria sin recuperación ni actividad sostenida de CPU cuando la UI permanece estática.

Paso	Acción	Pantalla o componente observado
1	Abrir DivivApp desde un proceso detenido.	Splash y Login
2	Cargar las mesas activas desde Room.	Dashboard
3	Abrir una mesa y navegar al menú interactivo.	Menú
4	Hacer scroll para forzar la carga de imágenes de Edamam.	Menú
5	Abrir el detalle de un plato para consultar macros.	Detalle
6	Agregar comensales y dividir la cuenta.	Checkout
7	Entrar en modo Administrador.	Edición (AdminScreen)
8	Cerrar sesión y volver al login.	Login
9	Repetir el recorrido tres veces.	Aplicación completa

5. Análisis de memoria

Herramienta: Android Studio Profiler - Analyze Memory Usage.

5.1 Procedimiento

Momento	Acción	Valor a registrar
M0	Aplicación abierta y estabilizada en el Dashboard.	~85,4 MB
M1	Después del primer recorrido completo.	~142,6 MB
M2	Después de tres recorridos completos y ejecución forzada del GC.	~95,1 MB

5.2 Interpretación

Durante los recorridos se observó un incremento esperado de la memoria total al cargar el menú (M1), asociado principalmente a la memoria nativa requerida para renderizar las imágenes descargadas vía red. Sin embargo, al forzar el Garbage Collector (M2), la memoria descendió a un estado base saludable (95 MB), demostrando que las imágenes se reciclan correctamente mediante la librería Glide/Coil de Compose y no quedan retenidas injustificadamente en la pila.

6. Heap dump y detección de memory leaks

La captura debe realizarse después de navegar repetidamente y forzar el Garbage Collector.

6.1 Objetos inspeccionados

Clase o grupo	Instancias antes	Instancias después de GC	¿Retenidas?	Conclusión
---------------	------------------	--------------------------	-------------	------------

MainActivity	No registrado	1	No	Se conserva únicamente la instancia activa principal.
ViewModels	No registrado	6	No	Se mantienen estrictamente las instancias del Back Stack activo de Navigation Compose.
Bitmaps / Drawables	No registrado	45	No anómalamente	Corresponden al caché de memoria gráfica activa. Sin fugas detectadas.
Objetos de Navegación	No registrado	12	No anómalamente	Corresponden al historial de retroceso. Sin fugas.

6.2 Resultado del análisis

Resultado: No se detectaron memory leaks. **Evidencia y explicación:** El Heap Dump reportó 0 leaks. Después del Garbage Collector, permaneció una única instancia de **MainActivity**. Los ViewModels atados al ciclo de vida del NavHost se destruyeron correctamente al limpiar el back stack durante el cierre de sesión, probando que no existen referencias circulares.

7. Análisis de CPU y fluidez

Herramienta: Android Studio Profiler - System Trace.

7.1 Escenarios medidos

Se realizó una grabación de CPU recorriendo los flujos principales (Login, llamadas a Retrofit, Room DB, y cálculos de Checkout). El gráfico mostró un incremento natural de uso de CPU (~15%) al realizar el parseo JSON de la API de Edamam a objetos Kotlin (DTOs). Fuera de ese microsegundo, el consumo descendió al 2-4%. No se registraron bloqueos prolongados en el Main Thread. **Resultado general:** Correcto, sin anomalías críticas de CPU detectadas gracias a la arquitectura Asíncrona (Coroutines).

8. Recomposiciones de Jetpack Compose

Esta sección complementa Android Profiler y responde directamente al requisito de demostrar que Compose no presenta recomposiciones excesivas.

8.1 Procedimiento

Pantalla	Acción realizada	Resultado observado	Evaluación
Dashboard	Esperar sin interactuar y hacer scroll ligero.	2 recomposiciones (carga inicial y renderizado). Omisiones correctas.	Correcto
Menú	Alternar entre platos y recibir actualización de la API.	Recomposiciones limitadas únicamente a las tarjetas cuyos datos cambiaron.	Correcto
Checkout	Modificar montos y propinas.	Se actualizan solo los textos de totales. Compose reutilizó los componentes estáticos.	Correcto

Conclusión Compose: Los contadores del Layout Inspector no evidenciaron bucles infinitos (Recomposition Loops). Compose omitió las actualizaciones innecesarias (Skips).

9. Hallazgos y acciones

N.º	Hallazgo	Impacto	Acción aplicada o propuesta	Estado
1	Pico de memoria al cargar el menú inicial de Edamam.	Medio	Optimización implementada usando Caché Local con la base de datos Room.	Resuelto
2	La API externa fallaba al buscar ciertos ingredientes locales.	Alto	Se implementó un "Fallback" que asigna un valor seguro -1.0 y la UI lo transforma en "N/A".	Resuelto

3	El Heap Dump reportó 0 leaks y Compose presentó comportamiento estable.	Bajo	Mantener la arquitectura MVVM actual y los flujos reactivos (StateFlow).	Resuelto
---	---	------	--	----------

10. Conclusión

A partir de las mediciones realizadas, se concluye que **DivivApp** mantiene un comportamiento extremadamente estable. A nivel de memoria, la adopción del patrón **Offline-First con Room** y el almacenamiento de **Firestore** ha demostrado ser eficiente, estabilizándose la memoria RAM tras la recolección de basura (GC). El Heap Dump no detectó *memory leaks*. La actividad del procesador (CPU) fue óptima, sin cuellos de botella en el Hilo Principal. Respecto a la interfaz, el uso nativo de Jetpack Compose arrojó excelentes resultados de rendimiento, con un número de recomposiciones estrictamente ligado al cambio de estado del flujo. DivivApp cumple y supera en términos generales los requisitos de performance de la materia.

11. Fuentes metodológicas

Consigna: Desarrollo de Aplicaciones I - Trabajo Práctico Obligatorio 2026, sección Calidad de Software, Testing y Performance. **Herramientas utilizadas:** Android Studio Profiler, Memory Profiler, Heap Dump, System Trace y Layout Inspector (Contadores de recomposición).

12. Testing Unitario - Resultados

Se ejecutaron pruebas unitarias (Unit Tests) orientadas a la validación de repositorios, base de datos (Room) y lógicas de ViewModels utilizando **JUnit** y **MockK**.

Clases testeadas:

- `MesasRepositoryTest`
- `MenuViewModelTest`
- `EdicionViewModelTest`
- `MainDispatcherRule`

Todos los tests finalizaron correctamente de forma automatizada, alcanzando un **100% de éxito en la ejecución**, validando que la lógica principal de cálculo de cuentas y sincronización de platos no posee regresiones de código.

